

Finite Automata in Haskell

V. Velkey, W. Vromen

June 2, 2023

Overview

- ▶ Theory overview: DFA-s, NFA-s, ϵ -NFA-s, Regular expressions
- ▶ DFA -s: implementation, evaluation of words
- ▶ NFA-s and ϵ -NFA-s: implementation, evaluation
- ▶ Powerset construction: ϵ -NFA to DFA
- ▶ RegExp: implementation, word generation
- ▶ RegExp to DFA
- ▶ DFA to RegExp

DFA -s: implementation, evaluation of words

Design choice: words with unknown symbols will be *rejected*.

```
type Symbol = Char
data DFA a = DFA { states :: [a]
                  , alphabet :: [Symbol]
                  , delta :: Symbol -> a -> a
                  , start :: a
                  , acceptstate :: [a]}

evaluate :: (Eq a, Ord a) => DFA a -> String -> Bool
evaluate df st = all ('elem' alphabet df) st && -- all characters in
    string are in the alphabet
    stateArr (start df) st 'elem' acceptstate df where
        stateArr q [] = q
        stateArr q (x:xs) = stateArr (delta df x q) xs
```

NFA implementation

Transitioning between lists of states: double recursion.

```
data NFA a = NFA { statesNF :: [a]
                  , alphabetNF :: [Symbol]
                  , deltaNF :: Symbol -> a -> [a]
                  , startNF :: a
                  , acceptstateNF :: [a]}

evaluateNF :: Eq a => NFA a -> String -> Bool
evaluateNF nf st = all ('elem' alphabetNF nf) st && -- captures elements
  of string in alphabet
    any ('elem' acceptstateNF nf) (stateArrNF' [startNF nf] st)
      where
        stateArrNF' qs [] = qs
        stateArrNF' [] _ = []
        stateArrNF' [q] (x:xs) = stateArrNF' (deltaNF nf x q)
          xs --recursion on string
        stateArrNF' (q:qs) (x:xs) = stateArrNF' [q] (x : xs) '
          union' stateArrNF' qs (x : xs) --recursion on
          statespace
```

RegExp implementation: data type

```
data RegExp = Empty | Epsilon | R [Symbol] | Union RegExp RegExp  
            | Star RegExp | Con RegExp RegExp | Plus RegExp  
deriving (Show, Eq)
```

Design choices:

RegExp implementation: data type

```
data RegExp = Empty | Epsilon | R [Symbol] | Union RegExp RegExp
            | Star RegExp | Con RegExp RegExp | Plus RegExp
  deriving (Show, Eq)
```

Design choices:

- ▶ not split empty from non-empty expressions
- ▶ Epsilon versus R []
- ▶ R [Symbol] versus R Symbol

For convenience:

```
regExpUnion :: [RegExp] -> RegExp
regExpUnion = simplify . foldr Union Empty
```

RegExp but prettier

```
pRegExp :: RegExp -> String
pRegExp Empty = ""
pRegExp Epsilon = "R e"
pRegExp (R []) = "R e"
pRegExp (R xs) = show xs
pRegExp (Union r1 r2) = "(" ++ pRegExp r1 ++ "|" ++ pRegExp r2 ++ ")"
pRegExp (Star r) = "(" ++ pRegExp r ++ ")*"
pRegExp (Con r1 r2) = pRegExp r1 ++ pRegExp r2
pRegExp (Plus r) = "(" ++ pRegExp r ++ "+)"

ppRegExp :: RegExp -> IO ()
ppRegExp = print . pRegExp
```

ghci> ppRegExp (Con (Union (R "53") (R [])) (Star (R "19")))

then becomes

"("53"|R e)("19")*"

RegExp Arbitrary

```
instance Arbitrary RegExp where
  arbitrary = sized randomReg where
    randomReg :: Int -> Gen RegExp
    randomReg 0 = R <$> elements ["0", "1"]
    randomReg n = oneof [ R <$> elements ["0", "1"]
      , Star <$> randomReg (n `div` 8)
      , Plus <$> randomReg (n `div` 8)
      , Con <$> randomReg (n `div` 8)
      , <*> randomReg (n `div` 8)
      , Union <$> randomReg (n `div` 8)
      , <*> randomReg (n `div` 8)
      , return Epsilon ]
```

We chose 'div' 8 to avoid dealing with monstrosities

RegExp to Generate Words

```
generateString :: RegExp -> Gen String
generateString = ('generateString', 5) . simplify
  where generateString' :: RegExp -> Int -> Gen String
        generateString' Empty _ = error "cannot generate from empty
            language"
        generateString' Epsilon _ = return ""
        generateString' (R xs) _ = return xs
        generateString' (Union r1 r2) n = oneof [generateString' r1 n,
            generateString' r2 n]
        generateString' (Con r1 r2) n = do
            w <- generateString' r1 n
            v <- generateString' r2 n
            return (w ++ v)
        generateString' (Star _) 0 = return ""
        generateString' (Star r) n = oneof [generateString' Epsilon n,
            generateString' (Con r (Star r)) (n - 1)]
        generateString' (Plus r) n = generateString' (Con r (Star r)) n
```

Size and Error

```
generateString = ('generateString' ' 5) . simplify
...
generateString' (Star r) n = oneof [generateString' Epsilon n,
                                     generateString' (Con r (Star r)) (n - 1)]
```

- The 5 is there to ensure that we don't generate infinite strings.

Size and Error

```
generateString = ('generateString' 5) . simplify
...
generateString' (Star r) n = oneof [generateString' Epsilon n,
                                     generateString' (Con r (Star r)) (n - 1)]
```

- ▶ The 5 is there to ensure that we don't generate infinite strings.

```
generateString' Empty _ = error "cannot generate from empty language"
```

- ▶ The error is a consequence of not splitting empty and non-empty expressions
- ▶ However, the error is only thrown when you do `generateString Empty`.
- ▶ That is because of `simplify`

And Simplify?

- ▶ Minimising regular expressions is (allegedly) PSPACE-hard
- ▶ So we did something a bit simpler...

And Simplify?

- ▶ Minimising regular expressions is (allegedly) PSPACE-hard
- ▶ So we did something a bit simpler...

```
simplify :: RegExp -> RegExp
simplify r | r == simplify' r = r
| otherwise = simplify $ simplify' r
where
simplify' :: RegExp -> RegExp
simplify' Empty = Empty
simplify' Epsilon = Epsilon
simplify' (R xs) = R xs
simplify' (Con Empty Empty) = Empty
simplify' (Con Epsilon Epsilon) = Epsilon
simplify' (Con _ Empty) = Empty
simplify' (Con Empty _) = Empty
simplify' (Con r' Epsilon) = simplify' r'
simplify' (Con Epsilon r') = simplify' r'
simplify' (Con r1 r2) = Con (simplify' r1) (simplify' r2)
simplify' (Union Empty Empty) = Empty
simplify' (Union Empty r') = simplify' r'
simplify' (Union r' Empty) = simplify' r'
simplify' (Union Epsilon Epsilon) = Epsilon
simplify' (Union r1 r2) | r1 /= r2 = Union (simplify' r1) (simplify' r2)
| otherwise = simplify' r1
```

RegExp to DFA

- ▶ For each RegExp R there is a DFA that accepts exactly all words from R .

RegExp to DFA

- ▶ For each RegExp R there is a DFA that accepts exactly all words from R .
- ▶ But how?

RegExp to DFA

- ▶ For each RegExp R there is a DFA that accepts exactly all words from R .
- ▶ But how?
- ▶ Recursively!
- ▶ First we convert regular expressions to ϵ -NFAs
- ▶ Then we just use the power set construction

RegExp to ϵ -NFA: the Base Cases

```
regExpToENFA :: RegExp -> ENFA Int
regExpToENFA Empty      = ENFA [1] [] (\_ _ -> []) [] 1 []
regExpToENFA Epsilon    = ENFA [1] [] (\_ _ -> []) [] 1 [1]
regExpToENFA (R [])     = ENFA [1] [] (\_ _ -> []) [] 1 [1]
regExpToENFA (R xs)     = ENFA [0..length xs] (nub xs) delta2 [] 0 [length xs]
    ]      where
    delta2 symbol state | state >= length xs || state < 0 = []
    | xs !! state == symbol = [state + 1]
    | otherwise = []
```

RegExp to ϵ -NFA: The Recursion

```
regExpToENFA (Union r1 r2) = regExpToENFA r1 'unionENFA' makeDisjoint (
    regExpToENFA r1) (regExpToENFA r2)
regExpToENFA (Star r) = starENFA (regExpToENFA r)
regExpToENFA (Con r1 r2) = regExpToENFA r1 'concatENFA' makeDisjoint (
    regExpToENFA r1) (regExpToENFA r2)
regExpToENFA (Plus r) = regExpToENFA r 'concatENFA' makeDisjoint (
    regExpToENFA r) (starENFA (regExpToENFA r))
```

Combining ϵ -NFA-s: Union

```
unionENFA :: ENFA Int -> ENFA Int -> ENFA Int -- Use only if states are
disjoint
unionENFA n1 n2 = ENFA states'' alphabet2 delta'' epT start'' accept where
  states'' = start'' : statesENF n1 ++ statesENF n2
  alphabet2 = alphabetENF n1 `union` alphabetENF n2
  delta'' sym st = deltaENF n1 sym st ++ deltaENF n2 sym st
  epT = epTrans n1 ++ epTrans n2 ++ [(start'', startENF n1), (start
'', startENF n2)]
  start'' = maximum (statesENF n2 ++ statesENF n1) + 1
  accept = acceptstateENF n1 ++ acceptstateENF n2
```

Combining ϵ -NFA-s: Union

```
unionENFA :: ENFA Int -> ENFA Int -> ENFA Int -- Use only if states are
disjoint
unionENFA n1 n2 = ENFA states'' alphabet2 delta'' epT start'' accept where
  states'' = start'' : statesENF n1 ++ statesENF n2
  alphabet2 = alphabetENF n1 `union` alphabetENF n2
  delta'' sym st = deltaENF n1 sym st ++ deltaENF n2 sym st
  epT = epTrans n1 ++ epTrans n2 ++ [(start'', startENF n1), (start
'', startENF n2)]
  start'' = maximum (statesENF n2 ++ statesENF n1) + 1
  accept = acceptstateENF n1 ++ acceptstateENF n2
```

- Concat and Star are handled similarly
- Disjoint?

```
ghci> regExpToENFA (Union (R "0") ( R "1"))
ENFA([4,0,1,2,3],"01",["d('0',4) = []","d('0',0) = [1]","d('0',1) =
[]","d('0',2) = []","d('0',3) = []","d('1',4) = []","d('1',0) =
[]","d('1',1) = []","d('1',2) = [3]","d('1',3) =
[]"],[(4,0),(4,2)],4,[1,3])
```

DFA to RegExp

- ▶ And now for the fun part!

DFA to RegExp

► And now for the fun part!

```
dfaToRegExp :: Ord a => DFA a -> RegExp
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (
    length $ states dfaInt) | f <- acceptstate dfaInt ]
    where dfaInt = (makeIntDFA . minimizeDFA) dfa
```

DFA to RegExp

- And now for the fun part!

```
dfaToRegExp :: Ord a => DFA a -> RegExp
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (
    length $ states dfaInt) | f <- acceptstate dfaInt ]
    where dfaInt = (makeIntDFA . minimizeDFA) dfa
```

- The idea is as follows:
- First we make the automaton as small as possible.

DFA to RegExp

- And now for the fun part!

```
dfaToRegExp :: Ord a => DFA a -> RegExp
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (
    length $ states dfaInt) | f <- acceptstate dfaInt ]
    where dfaInt = (makeIntDFA . minimizeDFA) dfa
```

- The idea is as follows:
- First we make the automaton as small as possible.
- Then we rename the states a little (to consecutive Int [1..length \$ states dfa]).

DFA to RegExp

- And now for the fun part!

```
dfaToRegExp :: Ord a => DFA a -> RegExp
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (
    length $ states dfaInt) | f <- acceptstate dfaInt ]
    where dfaInt = (makeIntDFA . minimizeDFA) dfa
```

- The idea is as follows:
- First we make the automaton as small as possible.
- Then we rename the states a little (to consecutive Int [1..length \$ states dfa]).
- Then the magic happens.

Minimising a DFA: Brzozowski's algorithm

- ▶ Reversing the DFA
- ▶ Cutting out unreachable states
- ▶ Reverse it back
- ▶ Cut out unreachable states again

```
reverseDFA :: Ord a => DFA a -> DFA [Int]
reverseDFA d = enfaToDFA' $ ENFA sts alph delt ep st acc where
e = singleAccept $ cutDFA d
sts = statesENF e
alph = alphabetENF e
delt sym s = [t | t <- statesENF e, s `elem' deltaENF e sym t]
ep = [(s, t) | (t,s) <- epTrans e ]
st = head $ acceptstateENF e
acc = [startENF e]
```

The Magic

```
rijk :: DFA Int -> Int -> Int -> Int -> RegExp
rijk dfa i j 0 | i == j = regExpUnion $ Epsilon : labels
| null labels = Empty
| length labels == 1 = head labels
| otherwise = foldr Union (head labels) (tail labels)
where labels = [R [x] | x <- alphabet dfa, delta dfa x i == j]
rijk dfa i j k = Union (rijk dfa i j (k-1)) (Con (rijk dfa i k (k-1)) (Con
    (Star $ rijk dfa k k (k-1)) (rijk dfa k j (k-1))))
```

- ▶ $rijk$ stands for R_{ij}^k .
- ▶ i and j are indices of states (hence the renaming).
- ▶ The k indicates that we only consider states between i and j of index at most k .
- ▶ R_{ij}^0 thus covers all paths that do not pass through states.
- ▶ $R_{ij}^k := R_{ij}^{k-1} + R_{ik}^{k-1}(R_{kk}^{k-1})^*R_{kj}^{k-1}$

The Magic: continued

$$R_{ij}^k := R_{ij}^{k-1} + R_{ik}^{k-1}(R_{kk}^{k-1})^* R_{kj}^{k-1}:$$

- ▶ Suppose we have a path from i to j that does not visit states higher than k .
- ▶ R_{ij}^{k-1} covers the option that the path does not visit k .
- ▶ If the path does visit k , then:
- ▶ R_{ik}^{k-1} covers the path from i to k .
- ▶ $(R_{kk}^{k-1})^*$ covers all (already found) paths from k to itself.
- ▶ R_{kj}^{k-1} covers the path from j to k .

The Magic: continued

$$R_{ij}^k := R_{ij}^{k-1} + R_{ik}^{k-1}(R_{kk}^{k-1})^* R_{kj}^{k-1}:$$

- ▶ Suppose we have a path from i to j that does not visit states higher than k .
- ▶ R_{ij}^{k-1} covers the option that the path does not visit k .
- ▶ If the path does visit k , then:
- ▶ R_{ik}^{k-1} covers the path from i to k .
- ▶ $(R_{kk}^{k-1})^*$ covers all (already found) paths from k to itself.
- ▶ R_{kj}^{k-1} covers the path from j to k .
- ▶ Hence R_{ij}^k is the regular expression covering all paths between i and j not visiting states with index higher than k .

The Magic: the End

recall:

```
dfaToRegExp dfa = simplify $ regExpUnion [rijk dfaInt (start dfaInt) f (
    length $ states dfaInt) | f <- acceptstate dfaInt ]
```

- ▶ i is the initial state
- ▶ j is an accepting state
- ▶ k is the amount of states
- ▶ We take the union of R_{ij}^k for all accepting states j .

Example

```
zeroStart:: DFA Int
zeroStart = DFA [0,1,2] ['0', '1'] deltazero 0 [1] where
    deltazero:: Symbol -> Int -> Int
    deltazero char st
    | st == 0 && char == '0' = 1
    | st == 0 && char == '1' = 2
    | st == 1 = 1
    | st == 2 = 2
    | otherwise = -1
```

```
ghci> ppRegExp (dfaToRegExp zeroStart)
"("0"|"0"((R e|("0"|"1")))*(R e|("0"|"1")))"
```

Design choice: ϵ -transitions represented as list of tuple - ease with transitive closure

```
data ENFA a = ENFA { statesENF :: [a]
    , alphabetENF :: [Symbol]
    , deltaENF :: Symbol -> a -> [a]
    , epTrans :: [(a, a)]
    , startENF :: a
    , acceptstateENF :: [a] }

rtClose :: (Eq a, Ord a) => ENFA a -> [a] -> [a]
rtClose nf qs = sort (unionL [[p | p <- statesENF nf , (q, p) 'elem'
    trClose rcl ] | q <- qs] ) where
    rcl = refClose (statesENF nf) (epTrans nf)
```


The powerset construction - definition

- ▶ Take NFA $N = (Q_n, \Sigma, \delta_n, q_0, F_n)$
- ▶ Powerset construct: $D = (\mathcal{P}(Q_n), \Sigma, \delta, S, F_D)$
 - ▶ States: subsets of Q_n
 - ▶ Alphabet: same (of course)
 - ▶ transition function: $\delta_D(R, \sigma) = \cup_{s \in R} \delta_N(s, \sigma)$
 - ▶ Start state: singleton with previous start state
 - ▶ Accept states: $\{R \subset Q_N : S \cap F_n \neq \emptyset\}$
- ▶ For ϵ -NFA:
Similar idea, but at each step, need to take ϵ -closure at steps
 - ▶ Start state $\text{ecl}(\{q_0\})$
 - ▶ transition function: $\delta_D(R, \sigma) = \text{ecl}(\cup_{s \in R} \delta_N(s, \sigma))$